

Capstone Design [202601-ICE/CSE4020]

Project #2: TurtleBot3 AutoRace Lane-Following Speed Challenge

Team: Donatello

U2210088	Sarvar Islamov (Team Leader)
U2210013	Fuzaylkhon Abdurakhimov
U2210077	Gayday Aleksey
U2210086	Inomjonov Javohirbek
U2210099	Kalimullin Rinat
U2210122	Shovkatjon Komilov
U2210251	Javohirbek Xatamov

May 2026

1 Approach and Architecture

Our project was based on the official `turtlebot3_autorace_2020` ROS package provided by ROBOTIS. The baseline system was extended and heavily tuned for the physical racing track used during the demonstration.

The lane-following pipeline consists of several stages:

1. Camera image acquisition from the TurtleBot3 RGB camera
2. HSV-based color filtering for yellow, white, and red objects
3. Morphological filtering to remove image noise
4. Lane extraction using connected-component analysis
5. Lane center estimation
6. PD-controller-based steering correction
7. Velocity control and state-based mission handling

The system was implemented in Python using ROS Noetic, OpenCV, and CvBridge. The robot continuously subscribed to the camera topic and processed incoming frames in real time.

The architecture used several operational states:

- **LANE_YELLOW** — standard yellow-lane tracking mode

- **LANE_WHITE** — dashed highway lane tracking mode
- **TURN_LEFT** — autonomous turning state
- **SEEK_RED** — searching for the parking zone
- **PARK** — parking alignment phase
- **FINAL_PUSH** — final forward adjustment
- **DONE** — mission completed

The robot used HSV filtering to isolate lane colors under real lighting conditions. The lower portion of the image was selected as the Region of Interest (ROI) to reduce computational load and improve detection stability.

Connected-component analysis was used instead of traditional Hough line detection because it provided more stable lane tracking under curves and partial lane visibility.

A proportional-derivative (PD) controller converted lane center error into steering commands. Additional smoothing windows were implemented for both target positions and steering errors to reduce oscillation and jitter.

The robot also used adaptive speed control. When the steering error increased during sharp turns, the linear velocity was automatically reduced to improve stability and prevent line crossings.

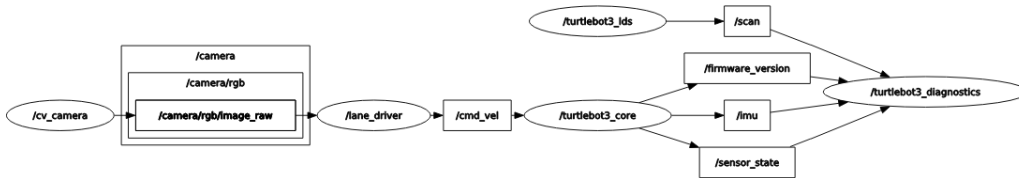


Figure 1: Lane-Following Pipeline and ROS Architecture

2 Parameter Choices

Extensive parameter tuning was required because the physical environment differed significantly from simulation conditions.

2.1 HSV Thresholds

The following HSV thresholds were selected after repeated testing under the classroom lighting conditions.

Yellow Lane Detection

- Hue: 23 – 65
- Saturation: 40 – 255
- Value: 117 – 255

These values allowed stable detection of the yellow boundary lines while minimizing false positives from reflections and floor textures.

White Lane Detection

- Hue: 0 – 179
- Saturation: 0 – 50
- Value: 208 – 255

The white dashed highway lines required high brightness filtering because the physical track lighting produced strong reflections.

Red Parking Zone Detection

- Hue: 163 – 179
- Saturation: 83 – 255
- Value: 85 – 255

These parameters allowed reliable parking area recognition during the final mission phase.

2.2 Control Parameters

The following PD controller values produced the most stable lane following:

Parameter	Value
Kp	0.004
Kd	0.006
Kp_WHITE	0.003
Kd_WHITE	0.005
Kp_RED	0.005
MAX_ANGULAR	1.5

Table 1: PD Controller Parameters

Higher proportional gains caused oscillation during curves, while lower values produced delayed steering responses. The derivative term helped smooth corrections and reduced overshooting.

2.3 Speed Parameters

Parameter	Value
MAX_SPEED	0.10 m/s
MIN_SPEED	0.04 m/s
WHITE_SPEED	0.10 m/s
SEEK_RED_SPEED	0.07 m/s
PARK_APPROACH_SPEED	0.04 m/s

Table 2: Velocity Parameters

We intentionally selected conservative speeds because stability and zero line crossings were prioritized over extremely aggressive racing.

2.4 Region of Interest

The parameter:

$$ROI_TOP_RATIO = 0.55$$

allowed the robot to process only the lower 45% of the image. This improved performance by focusing on the most relevant road area and reducing noise from distant objects.

3 Challenges and Solutions

3.1 Camera Angle and Visibility Problems

One major issue was the camera orientation. Initially, the camera was facing too far forward. Although it could see approximately 20 meters ahead, the lane markings appeared unclear and unstable due to perspective distortion and image noise.

To solve this problem, we adjusted the camera angle downward. After the modification, the camera viewed a smaller area of approximately 10 meters ahead, but the lane markings became significantly clearer and more stable. This improved lane detection consistency and reduced steering oscillations.

3.2 Camera Calibration Pipeline Problems

During intrinsic camera calibration, several problems were discovered in the original launch configuration.

Problem 1 — Unnecessary Republish Node

The original calibration pipeline used an `image_transport republish` node intended for compressed image streams. However, the `cv_camera` driver already published raw images directly.

As a result, the republish node waited for compressed topics that never existed.

The solution was to completely remove the unnecessary republish node from the launch file.

Problem 2 — Incorrect Camera Namespace

The original launch file incorrectly passed the full topic path to the calibration tool:

```
camera:=/camera/camera_info
```

However, `cameracalibrator.py` expects a namespace prefix instead of a full topic. The correct fix was:

```
camera:=/camera/rgb
```

This allowed the calibration node to correctly locate the required camera topics.

Problem 3 — Incorrect Image Rectification Order

The original pipeline rectified images before calibration using `image_proc`. This was logically incorrect because the calibration process requires the original distorted image in order to estimate lens distortion parameters.

Feeding already-rectified images produced incorrect calibration results.

The solution was to remove `image_proc` from the calibration pipeline so that the calibration tool received the raw distorted image directly from the camera.

3.3 Lane Detection Noise

Under different lighting conditions, HSV filtering occasionally detected floor reflections and shadows as lane markings.

To improve stability, morphological opening and closing operations were added after HSV masking. This significantly reduced noise and improved connected-component detection accuracy.

3.4 Sharp Curves and Oscillation

At higher speeds, the robot occasionally oscillated during sharp curves because steering corrections became too aggressive.

This was solved by:

- Reducing proportional gain values
- Adding derivative control
- Applying smoothing windows to steering errors
- Dynamically reducing speed during large steering corrections

These improvements produced smoother trajectories and prevented line crossings.

4 Results

The TurtleBot3 successfully completed the entire physical track autonomously without manual intervention.

- **Lap Time:** 04:11.22
- **Line Crossings Observed:** 0
- **Mission Status:** Successful full-track completion

The robot maintained stable lane following throughout the race and completed all required stages successfully.

4.1 Simulation vs Physical Performance

A direct comparison between Gazebo simulation and physical performance could not be fully performed because the available laptops were not powerful enough to run the simulation smoothly with reliable timing measurements.

As a result, most optimization and tuning work was conducted directly on the physical robot and track environment.

Despite this limitation, the physical robot demonstrated stable and consistent real-world performance.

5 Team Contributions

All team members contributed equally to the development, testing, debugging, and optimization of the project.

- **Sarvar Islamov (Team Leader)** — coordinated the project workflow, supervised integration, and participated in parameter tuning and testing.
- **Fuzaylkhon Abdurakhimov** — worked on ROS setup, package configuration, and calibration debugging.
- **Gayday Aleksey** — contributed to control tuning, debugging, and physical testing iterations.
- **Inomjonov Javohirbek** — participated in lane-detection implementation and HSV threshold optimization.
- **Kalimullin Rinat** — contributed to visualization, debugging tools, and runtime testing.
- **Shovkatjon Komilov** — participated in architecture integration, documentation, and testing coordination.
- **Javohirbek Xatamov** — worked on hardware preparation, camera positioning, and validation testing.

All members actively participated in testing sessions, troubleshooting, and performance optimization.

6 YouTube Video Links

Computer Screen Recording

<https://youtu.be/SDK73mEr38g>

Physical Robot Recording

<https://youtu.be/nZOmZpWM95w>

7 References

- https://github.com/ROBOTIS-GIT/turtlebot3_autorace_2020
- https://emanual.robotis.com/docs/en/platform/turtlebot3/autonomous_driving/
- https://docs.opencv.org/4.x/df/d9d/tutorial_py_colorspaces.html
- http://wiki.ros.org/image_transport
- <https://emanual.robotis.com/docs/en/platform/turtlebot3/bringup/>